AETHER_SHIELD

Deterministic HTTP Security Header Analyzer & Multi-Stack Remediation Engine

Project Architect:	Arya Tiwari (CSE Student, Bennett University, Greater Noida, India)
System Class:	Sec-Ops Deterministic HTTP Response Header Auditor & Config Synthesizer
Standard Protocols:	RFC-9110 (HTTP), RFC-6797 (HSTS), W3C CSP Level 3, SARIF v2.1.0
Core Repositories:	https://github.com/httpsarya/AETHER_SHIELD (Live: https://httpsarya.github.io/AETHER_SHIELD/)
Primary Purpose:	Complete technical guide & model Q&A; for Academic Viva Defense & Technical Audits

1. Executive Summary & Problem Statement

Modern web applications are increasingly vulnerable to client-side attacks, including Cross-Site Scripting (XSS), Clickjacking (UI Redressing), MIME-confusion sniffing, SSL stripping, and cross-origin data leaks. While developers invest heavily in application-layer defenses, HTTP security headers serve as the critical, standardized contract between the server and the user's browser, enforcing security boundaries directly at the networking protocol level.

The Problem: Over 70% of enterprise and consumer web applications fail to configure security headers correctly. Configuration errors stem from complex directive syntax (notably in Content Security Policy Level 3), stack-specific config idiosyncrasies, fear of breaking legacy third-party scripts, and the lack of deterministic, automated scoring tools in CI/CD pipelines.

The AETHER_SHIELD Solution: AETHER_SHIELD is an end-to-end cybersecurity telemetry platform that provides real-time, deterministic 0–100 rubric auditing, Server-Side Request Forgery (SSRF) safe target ingestion, redirect hop tracking, baseline regression diffing, batch multi-target auditing, and instant copy-paste configuration synthesis for 6+ web server stacks.

2. System Architecture & Technical Foundations

2.1 Technology Stack

Layer	Technology Selection	Architectural Rationale
Frontend UI	React 18, TypeScript, Vite 6	High performance, typed interfaces, sub-second HMR, and zero runtime bloat.
Design System	Vanilla CSS (Stitch Theme Tokens)	Obsidian & electric cyan cyber aesthetic; eliminates Tailwind bundle overhead.
Ingestion Engine	CORS Safe Ingestion Pipeline + RFC-9110 Parser	Multi-tier header fetcher supporting browser client, proxy fallback, and backend proxy.
CLI & Automation	Python 3.12 (urllib / requests / argparse)	Zero-dependency terminal execution with exit code assertions for CI/CD pipelines.
Report Standards	SARIF v2.1.0 & JSON Specification	OASIS standard interchange format for native GitHub Code Scanning integration.

2.2 SSRF Guard & Network Abuse Prevention

A critical vulnerability in web-based scanners is **Server-Side Request Forgery (SSRF)**, where attackers attempt to use the scanner as an internal proxy to query internal network services or cloud metadata. AETHER_SHIELD implements defense-in-depth SSRF protection:

• **RFC-1918 Private IP Filter:** Blocks private subnets (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16).

• **Loopback & Link-Local Filter:** Denies loopback addresses (127.0.0.0/8, ::1) and link-local ranges (169.254.0.0/16).

• **Cloud Metadata Shield:** Explicitly blocks AWS/GCP/Azure instance metadata endpoints (e.g., 169.254.169.254, metadata.google.internal).

• **DNS Single-Resolution & IP Pinning:** Resolves target DNS once and pins the destination IP across all HTTP redirect hops to eliminate DNS rebinding (TOCTOU) attacks.

2.3 The Deterministic 0–100 Scoring Rubric

Unlike arbitrary grading tools, AETHER_SHIELD employs a transparent, weighted mathematical model evaluating security posture across 7 key vectors:

Security Header Vector	Max Points	Evaluation Criteria & Penalty Rules
Content-Security-Policy (CSP)	25 pts	Full points for strict script-src / nonces / hashes. Deductions: -10 for 'unsafe-inline', -8 for 'unsafe-eval', -10 for wildcards (*).
Strict-Transport-Security (HSTS)	20 pts	Full points for max-age >= 31536000 + includeSubDomains + preload. Deductions: -5 for max-age < 1 yr, -5 for missing subdomains.
X-Frame-Options / frame-ancestors	15 pts	15 pts for DENY or frame-ancestors 'none'. 10 pts for SAMEORIGIN. 0 pts if framing is unrestricted (Clickjacking risk).
X-Content-Type-Options	10 pts	10 pts for 'nosniff'. 0 pts if missing (prevents MIME confusion and script execution from user uploads).
Referrer-Policy	10 pts	10 pts for strict-origin-when-cross-origin, same-origin, or no-referrer. Deductions: -6 for unsafe-url or no-referrer-when-downgrade.

Permissions-Policy	10 pts	10 pts for restricting geolocation, camera, microphone, and payment APIs. Deductions: -5 if unrestricted or legacy Feature-Policy.
Cross-Origin Isolation (COOP/COEP/CORP)	10 pts	10 pts for isolating process threads (Cross-Origin-Opener-Policy & Cross-Origin-Embedder-Policy) against Spectre/Meltdown.

Letter Grade Scale: A+ (95–100) | A (85–94) | B (70–84) | C (55–69) | D (40–54) | F (0–39).

3. Detailed Functional Walkthrough of All 9 Tools

Tool 1: Real-Time Scanner

The central audit engine of AETHER_SHIELD. Upon URL submission, the engine validates the target, executes DNS single-resolution, traces up to 5 HTTP redirect hops, and ingests raw HTTP response headers. The rubric calculates a 0–100 score and assigns a letter grade (A+ to F). The dashboard displays: (1) Tactical Grade Gauge, (2) Security Vector Breakdown, (3) Pass/Warn/Fail Finding Cards with OWASP citations, (4) Redirect Chain with IP and latency metrics, and (5) Raw Headers Inspector.

Tool 2: Policy Generator (CSP Constructor)

An interactive Content-Security-Policy (CSP) visual builder. Allows security engineers to construct robust policies without syntax errors. Features directive toggles for default-src, script-src, style-src, img-src, connect-src, font-src, and frame-ancestors. Supports CSP Level 3 keywords ('strict-dynamic', nonces, hashes), and includes a 'Content-Security-Policy-Report-Only' toggle to enable safe telemetry testing before full enforcement.

Tool 3: Diff Benchmark Engine

A regression detection tool designed for pre-deployment auditing. Compares a Baseline URL (e.g., Production) against a Candidate URL (e.g., Staging). Computes the score delta (+/- points), alerts on weakened or removed headers, highlights newly implemented protections, and generates a diff matrix to prevent security regressions from reaching production.

Tool 4: Multi-Stack Remediation Guide

Eliminates the friction between identifying a vulnerability and fixing it. Generates tailored, production-ready configuration snippets for 6+ web server and framework stacks: (1) Nginx (add_header directives), (2) Apache 2.4 (mod_headers), (3) Caddy, (4) Cloudflare Workers / Transform Rules, (5) Express.js (Helmet middleware config), and (6) Python Django / Flask (SecurityMiddleware and Talisman).

Tool 5: Compare Tool

Enables side-by-side comparative inspection between two arbitrary websites (e.g., comparing your organization's posture against competitors, or comparing cloud providers). Renders dual grade gauges, comparative vector scores, and a side-by-side header presence matrix.

Tool 6: Bulk Audit Engine

Built for DevSecOps and penetration testers managing domain portfolios. Audits up to 25 target URLs concurrently with built-in concurrency throttling. Renders an aggregate letter grade distribution chart, average portfolio score, and enables one-click batch export to CSV and JSON.

Tool 7: Learn & Security Encyclopedia

An educational knowledge base explaining 12+ security headers in plain English. Includes RFC citations, attack models mitigated (XSS, Clickjacking, MIME sniffing, Protocol Downgrade), threat severity ratings, and interactive test buttons to scan real-world target implementations.

Tool 8: Scan History & Telemetry Store

A persistent client-side audit log powered by LocalStorage. Automatically records the last 30 scans with URL, timestamp, score, grade, and redirect metrics. Supports re-scanning with a single click, multi-scan selection for comparison, and history backup/export.

Tool 9: Developer API & CLI Documentation

Comprehensive technical documentation for programmatic automation. Details the REST API endpoints, request/response JSON schemas, and the standalone Python CLI tool (cli/main.py) which supports SARIF 2.1.0 output, customizable failure thresholds (--fail-under 80), and direct integration into GitHub Actions CI/CD workflows.

4. Academic Viva Defense Guide: 15 Core Questions & Answers

This section prepares the candidate for technical questioning by external examiners and professors during project defense.

Q1: Why are HTTP security headers necessary if web applications already implement application-level input validation?

A: Application-level validation protects server-side logic from malicious input, but HTTP security headers enforce browser-level security boundaries. For example, Content-Security-Policy (CSP) restricts what scripts can execute even if an attacker successfully injects XSS payloads into the DOM. Similarly, X-Frame-Options prevents Clickjacking regardless of application code, and HSTS enforces encrypted transport before the browser even sends the initial request. They represent defense-in-depth at the protocol level.

Q2: How does AETHER_SHIELD prevent Server-Side Request Forgery (SSRF) during scanning?

A: AETHER_SHIELD applies multi-tier SSRF protection: (1) It parses the destination hostname and resolves DNS to an IP address. (2) It validates that the IP does not fall into RFC-1918 private subnets (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), loopback (127.0.0.1), or link-local/cloud metadata ranges (169.254.169.254). (3) It pins the validated IP across redirect hops to prevent Time-of-Check to Time-of-Use (TOCTOU) DNS rebinding attacks.

Q3: What is the difference between CSP Level 2 and CSP Level 3 'strict-dynamic'?

A: In CSP Level 2, developers had to whitelist domain origins (e.g., script-src https://cdn.example.com). This approach proved brittle and vulnerable to JSONP bypasses. CSP Level 3 introduced 'strict-dynamic' with cryptographically random nonces (number used once). With 'strict-dynamic', trust is granted to a root script via a nonce, and any subsequent scripts dynamically created by that trusted root script are automatically trusted, eliminating the need for complex domain whitelists.

Q4: What is HSTS Preloading and what are the requirements for a domain to be eligible?

A: HSTS Preload is a browser feature where major browsers (Chrome, Firefox, Safari) hardcode domains that must ONLY ever be contacted over HTTPS. This eliminates the vulnerability window during a user's first visit (TOFU - Trust On First Use) where SSL stripping could occur. Requirements: (1) Valid TLS certificate, (2) Redirect all HTTP to HTTPS on port 80, (3) Serve HSTS header with max-age of at least 31536000 (1 year), (4) Include the 'includeSubDomains' directive, and (5) Include the 'preload' flag.

Q5: Why did you choose SARIF v2.1.0 instead of plain JSON for automated reports?

A: SARIF (Static Analysis Results Interchange Format) is an OASIS standard widely adopted by GitHub Code Scanning, GitLab CI, and Microsoft Azure DevOps. By outputting SARIF 2.1.0, AETHER_SHIELD integrates natively with GitHub Actions, allowing security findings to appear directly as inline code scanning alerts in pull requests with zero third-party plugins.

Q6: How does the deterministic scoring algorithm avoid false positives?

A: The algorithm evaluates explicit, deterministic rules based on official RFC specifications and OWASP standards rather than heuristic guessing. For example, if a header is present but contains unsafe directives (e.g., CSP with 'unsafe-inline' or 'unsafe-eval'), the rubric assigns partial credit with clear, explainable deduction reasons rather than binary pass/fail judgments.

Q7: What happens when an audited website has HTTP redirects (e.g. HTTP to HTTPS)?

A: AETHER_SHIELD's redirect tracer tracks each redirect hop sequentially (up to 5 hops). It records the HTTP status code (301, 302, 307, 308), the destination URL, and latency for each hop. The final security score is computed against the terminal response headers that the end user's browser actually receives, while also verifying that the redirect chain properly upgraded from HTTP to HTTPS.

Q8: How does the web client inspect headers of arbitrary public domains without getting blocked by browser CORS restrictions?

A: Modern browsers restrict web apps from reading raw response headers of cross-origin domains via client-side fetch. AETHER_SHIELD overcomes this using a robust multi-tier fallback pipeline: (1) It attempts direct inspection if CORS permits, (2) Fallback to a high-availability public header proxy (HackerTarget API), and (3) In production environments, requests route through the AETHER_SHIELD Python FastAPI backend, which executes server-side requests with full raw socket header visibility.

Q9: What is the difference between 'X-Frame-Options: DENY' and CSP 'frame-ancestors none'?

A: X-Frame-Options is an older header defined in RFC 7034 that only supports DENY, SAMEORIGIN, or ALLOW-FROM (deprecated). CSP 'frame-ancestors' is the modern W3C standard. When both are present, CSP frame-ancestors takes precedence according to modern browser specifications. However, serving both is considered best practice for backward compatibility with legacy browsers.

Q10: Why is 'X-Content-Type-Options: nosniff' so critical even if the server sets Content-Type?

A: Browsers historically implemented 'MIME sniffing' to guess file types if Content-Type was missing or deemed incorrect. An attacker could upload a file containing malicious JavaScript masked as an image (e.g., avatar.jpg). If the browser sniffed the file and executed it as JavaScript, it resulted in XSS. The 'nosniff' directive strictly forces the browser to honor the declared Content-Type, completely mitigating MIME-confusion attacks.

Q11: What is Permissions-Policy (formerly Feature-Policy) and what does it restrict?

A: Permissions-Policy allows site administrators to declare which hardware and browser APIs can be used by the page or embedded iframes. It allows explicitly disabling access to sensitive sensors and APIs such as geolocation, camera, microphone, accelerometer, payment, and autoplay, drastically reducing the attack surface if a third-party script or ad is compromised.

Q12: Describe the architecture of your Python CLI tool compared to the Web Application.

A: The CLI tool (cli/main.py) is a self-contained, dependency-free utility written in standard Python. It uses urllib and http.client to perform raw HTTP handshakes, inspect headers, evaluate the scoring rubric, and format results as ANSI terminal tables, JSON, or SARIF files. It returns standard UNIX exit codes (e.g., exit code 1 if score < threshold), making it plug-and-play in shell scripts and automated build pipelines.

Q13: How is state managed in the web frontend without a dedicated relational database?

A: The frontend utilizes React state combined with HTML5 LocalStorage for persistent client-side telemetry. Scans, comparative pairs, and custom CSP configurations are serialized as structured JSON in LocalStorage, supporting up to 30 persistent history items with zero server-side storage overhead, ensuring complete data privacy for users auditing internal domains.

Q14: What are Cross-Origin Isolation headers (COOP, COEP, CORP) and why are they evaluated?

A: Post-Spectre and Meltdown, microarchitectural side-channel attacks allowed malicious scripts to read memory across origins. Cross-Origin-Opener-Policy (COOP) and Cross-Origin-Embedder-Policy (COEP) place the web page into a separate browser process group, enabling safe access to high-precision timers (SharedArrayBuffer, performance.now()) without cross-origin memory leakage.

Q15: What are the primary future enhancements planned for AETHER_SHIELD?

A: Future roadmap milestones include: (1) Automated CSP violation report collector endpoint (report-to / report-uri ingest), (2) Cookie security audit (HttpOnly, Secure, SameSite=Strict/Lax flag evaluation), (3) TLS certificate cipher suite and key exchange validation, and (4) An official GitHub Action in the GitHub Marketplace.

5. Project Candidate Profile & Academic Endorsement

This project documentation and technical implementation were designed and developed independently by **Arya Tiwari** as part of the Computer Science & Engineering curriculum at **Bennett University**, Greater Noida, India.

Candidate Name:	Arya Tiwari
Institution:	Bennett University, Greater Noida, UP, India
Degree Program:	Bachelor of Technology (B.Tech) - Computer Science & Engineering
Specialization:	Cybersecurity, Cloud Infrastructure & Web Engineering
Professional Portfolio:	https://httpsarya.github.io/Arya_Tiwari_portfolio_12/
GitHub Profile:	https://github.com/httpsarya
LinkedIn:	https://www.linkedin.com/in/arya-tiwari-01ba84292/
Contact Email:	arya2tiwari@gmail.com

6. Technology Reference Checklist for Viva

Feature / Standard	Specification Reference	Status
HTTP Semantics	RFC 9110 (IETF Standard)	**Verified**
HSTS Policy	RFC 6797 (IETF Standard)	**Verified**
Content Security Policy	W3C Recommendation CSP Level 3	**Verified**
SARIF Interchange	OASIS SARIF v2.1.0	**Verified**
MIME Sniffing	WHATWG MIME Sniffing Standard	**Verified**
Cross-Origin Resource Policy	W3C Cross-Origin Embedder / Opener	**Verified**